

withdraw funds from the platform beyond the intended functionality. The former could be any typical software bug in the code. For example, consider a smart contract intended to escrow deposits from a particular ERC-20 from any user and transfer the entire balance to a lottery winner. The contract keeps track of how many tokens it has internally and uses that number as the amount when performing the transfer. The bug will belong here in our hypothetical contract. The internal number will, due to a rounding error, be slightly higher than the actual balance of tokens the contract holds. When it tries to transfer, it will transfer “too much,” and the execution will fail. Without a failsafe, the tokens are functionally locked within the protocol. Informally, these are known as “bricked” funds and cannot be recovered.

An economic exploit would be subtler. There would be no explicit failure in the logic of the code but rather an opportunity for an economically equipped adversary to influence market conditions to profit inappropriately at the contract’s expense. For example, consider a contract that takes the role of an exchange between two tokens and determines the price by looking at the exchange rate of another similar contract elsewhere on chain and offering that rate with a minor adjustment. (The other exchange is playing the role of a price oracle for this particular contract.) The possibility for an economic exploit arises when the oracle exchange has significantly lower liquidity compared with the primary exchange in the example. A financially equipped adversary can sell